GIR: A Persistent Intermediate Representation for Scalable Dynamic Graph Analytics

***Abstract*—Dynamic graphs demand both high-throughput structural updates and efficient downstream analytics. Existing methods are either *history-blind* as they discard prior computation after each update, incurring large repeated overhead or *query-dependent* as they reuse results but are tied to specific algorithms or parameters, offering no general reuse. Neither approach captures algorithm-independent structural invariants as a persistent, reusable layer. Inspired by compiler intermediate representations, we propose GIR, a query-agnostic *Graph Intermediate Representation* that decouples dynamic graph updates from downstream query execution. GIR maintains a three-level structural hierarchy of weakly connected components, strongly connected components with a condensation DAG, and biconnected components organized as block-cut trees. To keep GIR current under continuous updates, we design strategies to confine each repair to the minimum affected region. GIR further enables a two-level parallel execution model that combines task parallelism across independent blocks with data parallelism within each block. Experiments on seven real-world graphs show that GIR achieves 1.8× more update throughput than state-of-the-art methods. For graph analytics, GIR achieves an average of 1.3×, 2.7×, and 1.7× speedups for breadth-first search, single-source shortest path, and betweenness centrality, respectively.**

I. Introduction

The graph is a fundamental abstraction for modeling relationships and dependencies across diverse domains, including social and information networks, transportation systems, cybersecurity provenance, and software package ecosystems [1]–[3]. Among these, directed graphs are of particular interest as they capture asymmetric dependencies and causal flows that underpin critical analytics, such as breadth-first search (BFS) [4] for traversal, single-source and all-pairs shortest paths (SSSP/APSP) [5], [6] for routing and dependency, and betweenness centrality (BC) [7] for node importance.

Over the past years, graph processing systems and domain-specific frameworks have achieved substantial performance improvements for *static* graph analytics on both CPUs and GPUs [8]–[10]. However, real-world graphs are inherently *dynamic*, i.e., vertices and edges continuously appear and disappear as the underlying system evolves [11], [12]. At the same time, the users might issue *ad hoc* queries with changing sources, targets, and scopes. For example, a user may need to trace the blast radius of a newly disclosed vulnerability, recompute reachability after integrating new services, or update dependency analysis in an evolving software ecosystem [13]. These workloads combine continuous structural updates with repeated, query-driven traversals, *forming an end-to-end pipeline that must be efficient in both dimensions*.

Existing dynamic graph systems primarily focus on *update throughput*, i.e., how quickly structural changes can be incorporated into the graph [14]–[17]. However, update throughput only addresses part of the end-to-end workload. That is, after each structural change, downstream analytics such as BFS, SSSP, BC, and reachability often need to be recomputed, either partially or entirely. As graphs grow to billions of edges, such downstream graph analytics increasingly dominate the total runtime. *Therefore, optimizing update throughput alone is not enough, while reducing the cost of downstream graph computation under continuous evolution is equally critical.*

Two extremes of reusing prior computation. After a graph update, the results from previous computation do not simply become invalid. Consider running SSSP from source s on graph snapshot G, when G is updated to G', many of the shortest paths from s remain unchanged. A system that could identify which results are still valid and which need to be recomputed would avoid a large fraction of redundant work. However, the key question is, in what form should prior computation be retained to maximize reuse? Existing approaches fall into two extremes, where they either igore such previous computation information (history-blind system) or overly rely on them (query-dependent system).

(i) History-blind systems. Most dynamic graph systems, including Terrace [18], Spruce [19], and Aspen [20], treat each query independently. That is, after an update, such a system integrates the change into the graph representation and reruns the queried graph algorithm. These systems are generic to any queries, while they discard the prior computation after every update. For large graphs with frequent updates, such a system will incur large overhead due to repeated recomputation.

(ii) Query-dependent systems. A different class of systems try to reuse prior results more directly. For example, *Khanda et al.* designed a method by maintaining an SSSP tree rooted at a fixed source s after each query [21]. When the graph is updated, it identifies the subtree affected by the change and recomputes only that part. This approach can be very efficient when the same query is repeated from the same source. However, it is fundamentally tied to a specific source node and a specific algorithm. If the next query uses a different source, or asks for other algorithms (e.g., BFS) instead of SSSP, the cached tree provides no benefit. In practice, dynamic graph workloads are dominated by *ad hoc* queries with varying sources, targets, and algorithms [11]. Thus, such query-dependent systems offer little reuse in practice.

A compiler mindset for dynamic graph. The two extremes reveal a clear gap. That is, *we need a method that not only leverages historical computation results but also is not tied to any specific queries or algorithms*. Inspired by the intermediate

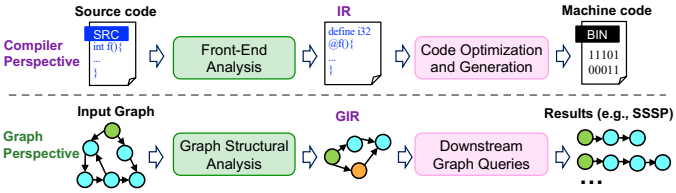


Fig. 1. *From compiler IR to Graph IR (GIR).* Compiler IR enables reusable optimizations across different programming languages. Similarly, GIR maintains reusable structural invariants across dynamic updates.

representation (IR) design in compiler [22], we propose a new IR for dynamic graphs, named graph IR (GIR). The goal is to capture ***reusable structural invariants*** that persist across dynamic updates and can benefit many algorithms, without being tied to any specific queries or algorithms.

Just as a compiler IR sits between front-end parsing and back-end optimization passes, GIR sits between graph updates and downstream analytics. As shown in Figure 1, this mirrors the compiler pipeline. That is, updates modify the graph structure (analogous to front-end), and downstream algorithms repeatedly operate on the result (analogous to optimizations).

Contribution. Building GIR under continuous, high-velocity updates with ad hoc query patterns is nontrivial. There are three challenges must be addressed. *(i) Update velocity*: low-latency GIR repair is required after each node/edge insertion or deletion. *(ii) Ad hoc queries*: the query source, target, and algorithm can change between requests. Thus, no query-specific state can be assumed. *(iii) Large scale graphs*: the graph could become large, e.g., with billion edges. Thus, the end-to-end computaiton, including update and query, needs to be efficient. To address such challenges, this paper presents GIR, a scalable dynamic graph analytics system based on a newly designed persistent intermediate representation. Particularly, we make four major contributions.

Graph intermediate representation (GIR). We devise GIR, a dynamic, query-agnostic structural layer for graphs. GIR integrates three fundamental primitives. First, *weakly connected components (WCCs)* partition the graph into independent regions, enabling algorithms to skip unreachable components and prune work at a coarse level. Second, *strongly connected components (SCCs)* and their condensation directed acyclic graph (DAG) capture directed reachability, reducing traversal to a DAG and avoiding redundant exploration of cycles. Third, *biconnected components (BiCCs)* and the block-cut tree expose articulation boundaries, allowing independent processing of subgraphs. These three primitives are chosen because they are algorithm-independent structural invariants that expose natural hierarchical boundaries many algorithms implicitly rediscover during computation.

Structure-aware incremental maintenance. We design an incremental maintenance method that keeps GIR structurally consistent under dynamic graph updates without full recomputation. *The core insight is that GIR's three-layer hierarchy enables a metadata-only classification of every edge insertion or deletion into one of five structurally distinct cases, each confining repair to the minimum necessary region.* We formalize this as a scope-prune-update strategy for single-edge updates and extend it to a filter-merge-update strategy for batch updates. Together, they eliminate redundant operations, merge overlapping maintenance regions, and enable parallel execution. Benefited from this, for the update with both insertions and deletions, on all the tested graphs, GIR achieves an average throuput of 25.4 million edges per second (MEPS), which is $1.8\times$ more than state-of-the-art methods.

GIR-enabled two-level parallel execution. Conventional dynamic graph systems rely primarily on data parallelism, i.e., all threads work on the same task simultaneously. GIR unlocks a second, complementary form of parallelism, i.e., task parallelism. As GIR explicitly decomposes the graph into independent structural units, multiple units can be processed as concurrent tasks, each without inter-task synchronization. Taking BFS as an example, we can prune their search space using the condensation DAG and then execute one task per BiCC block in parallel. With the additional task parallelism, on all the tested graphs, GIR achieves an average of $1.3\times$, $2.7\times$, and $1.7\times$ speedups over state-of-the-art methods for running BFS, SSSP, and betweenness centrality, respectively.

End-to-end evaluation. We evaluate GIR on seven real-world graphs covering social networks, citation graphs, and web graphs, with the largest one having 2.4 billion edges. We compared GIR with state-of-the-art dynamic graph computation methods, including Terrace [18], Aspen [20], and Spruce [19]. Besides the mixed (insertion and deletion update), on average, GIR achieves 22.6 MEPS for *insertion-only update*, which is $9.1\times$, $3.4\times$, and $2.7\times$ more than Spruce, Aspen, and Terrace, respectively. For *deletion-only update*, GIR achieves $8.5\times$, $4.4\times$, $3.4\times$ more than Spruce, Aspen, and Terrace, respectively. In addition, GIR shows strong and stable scalability across all tested datasets.

II. Background

Static graph representation. High-performance graph processing systems rely on compressed sparse formats such as CSR (Compressed Sparse Row) and CSC (Compressed Sparse Column) [23]. These formats store adjacency information in contiguous arrays. They form the foundation of major graph processing frameworks, including Ligra [8], Gunrock [9], and GraphIt [10]. However, CSR and CSC are not suitable for dynamic workloads as vertex/edge insertions/deletions require rebuilding contiguous arrays, introducing significant overhead.

Dynamic graph representation. To support evolving graphs, recent systems propose specialized in-memory structures that balance update cost and query efficiency. Terrace [18] targets skewed graphs and uses a hierarchical adjacency representation that adapts its storage to vertex degree, improving locality and update efficiency on power-law graphs [24]. Spruce [19] combines compact indexing with update-friendly adjacency organization, integrating dynamic buffers with CSR-like sorted blocks and a read-optimized concurrency protocol. Aspen [20] represents graphs using compressed purely functional trees and exposes immutable snapshot views that support concurrent readers across updates.

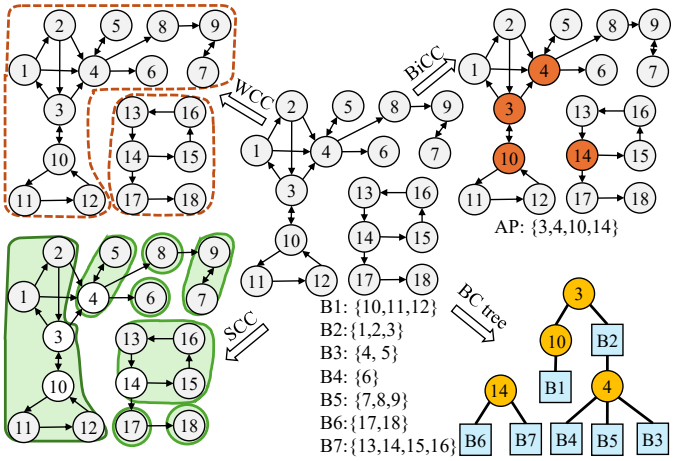


Fig. 2. Examples of weakly connected component (WCC), strongly connected component (SCC), biconnected component (BiCC), and block-cut (BC) tree.

Graph connectivity provides fundamental abstractions for graph topologies. *Weakly connected component (WCC)* is obtained by ignoring edge directions and grouping vertices that are reachable through undirected paths [25]. WCCs partition the graph into independent regions that remain disconnected even when edge directions are ignored. For the example in Figure 2, the subgraph containing vertices $\{1–12\}$ forms one WCC, while vertices $\{13–18\}$ form another. WCCs therefore represent the coarsest structural partition of the graph.

Strongly connected component (SCC) is defined as a maximal subset of vertices such that every pair of vertices is mutually reachable [26]. For the example in Figure 2, vertices $\{1–3, 10–12\}$, $\{13–16\}$ form a large SCC (shaded in green), while vertices $\{4, 5\}$ and $\{7, 9\}$ form small SCCs.

Biconnected component (BiCC) denotes a maximal subgraph that remains connected after removing any single vertex [27]. Vertices whose removal disconnects the graph are called articulation points (APs) [28]. For the example in Figure 2, $\{3, 4, 10, 14\}$ are identified as APs. The BiCC decomposition yields 7 blocks, including B_1 till B_7.

Block-cut (BC) tree organizes the blocks and APs into a tree structure. For example, AP vertex 10 connects blocks B_2 and B_1, while AP vertex 14 connects blocks B_6 and B_7. This tree structure explicitly exposes articulation boundaries within strongly connected regions.

III. Overview

Figure 3 overviews the end-to-end pipeline of our method on an exmaple graph. Particularly, GIR operates in three phases: GIR construction, structure-aware incremental maintenance, and GIR-enabled task parallel execution.

(i) For GIR construction, at the coarsest level, WCCs partition the graph into independent regions. Within each WCC, SCCs are contracted into a condensation DAG, eliminating redundant exploration of cyclic subgraphs, as shown in Figure 3(b). Within each SCC, BiCCs and their block-cut tree expose articulation boundaries, as shown in Figure 3(c). They enable independent processing of structurally isolated subgraphs. Together, these three layers make structural boundaries explicit and directly accessible, replacing boundaries that would otherwise be rediscovered implicitly at query time.

(ii) To keep GIR consistent under dynamic updates, we classify every edge insertion or deletion into one of five structurally distinct cases using only pre-maintained metadata. This confines each repair to the minimum necessary region via a scope-prune-update (SPU) strategy for single-edge updates and a filter-merge-update (FMU) strategy for batches.

(iii) GIR further unlocks task parallelism as a complement to conventional data parallelism. Because GIR decomposes the graph into independent structural units, multiple units can be processed as concurrent tasks without inter-task synchronization, offering a second dimension of parallelism that flat adjacency representations cannot provide.

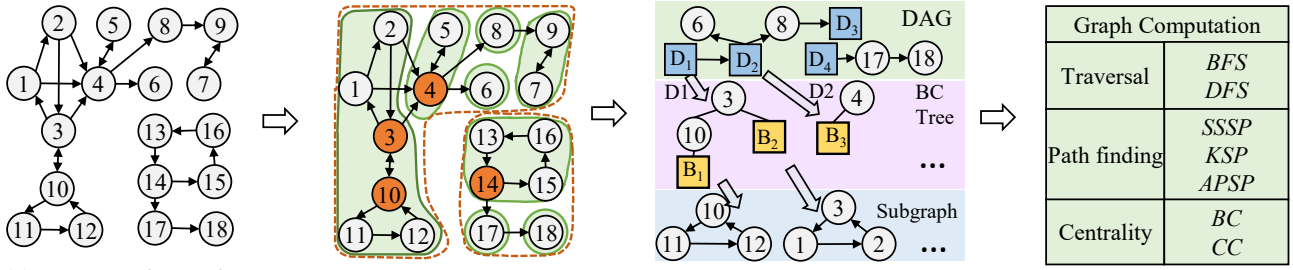
IV. A Novel Hierarchical Graph IR

GIR is built on a simple but powerful principle: *separate structural organization from algorithm execution*. However, not every structural property of a graph deserves to be maintained incrementally. Instead, we select invariants that satisfy three criteria simultaneously. First, the invariant must be *incrementally maintainable*, i.e., a single vertex/edge insertion or deletion should trigger only a local repair, not a full recomputation. Second, it must be *algorithm-independent*, meaning that it must reflect the structure of the graph itself, not the parameters of any specific query. Third, it must be *directly actionable*. That is, a downstream algorithm must be able to use it to skip work, not merely consult it as reference.

We observe that connectivity decomposition algorithms, including WCC, SCC (along with condensation DAG), and the BC tree, satisfy all three criteria. Together, they form a complete global-to-local hierarchy with no gap and no redundancy between levels. Motivated by this, we design the GIR as a three-level structural hierarchical layer.

Level 1: WCC is chosen as it provides the strongest possible pruning at zero traversal cost. No path of any kind can cross a WCC boundary. A query at any vertex is therefore entirely confined to that vertex's WCC. All other WCCs can be eliminated before a single edge is examined. WCC also has the simplest incremental maintenance rule of the three. That is, one insertion can only merge two WCCs, and one deletion can only split one. This makes the WCC layer a nearly free filter that benefits every source-rooted algorithm on the graph.

Level 2: SCC is used because it resolves the fundamental difficulty of directed traversal, i.e., cycles. In a directed graph, any strongly connected region is internally equivalent for reachability purposes. Every vertex in an SCC can reach every other vertex in the same SCC, so the SCC as a whole can be treated as a single unit. Contracting each SCC into a supernode and connecting them by the remaining inter-SCC edges produces the condensation directed acyclic graph (DAG). This transforms directed reachability into a DAG walk, eliminating all cycle-related redundancy. The DAG also yields a topological order that lets algorithms process SCCs level by level without backtracking. Incrementally, one edge insertion



(a) An example graph (b) Find WCC, SCC, BiCC (c) GIR Construction (d) GIR Application

Fig. 3. Overview of the pipeline. (a) shows an example graph, (b) decomposes it into WCCs, SCCs, and BiCCs, (c) constructs the GIR, and (d) performs downstream graph query analytics on GIR.

can at most merge SCCs by creating a new directed cycle, and one deletion can at most split one SCC. Both operations are local and do not affect the rest of the DAG.

Level 3: Block-cut tree is selected as it exposes the finest level of independent substructure within an SCC. Even inside a strongly connected region, the internal topology is not uniform. Some vertices are articulation points (APs), whose removal will disconnect the SCC into separate regions. Any path between vertices in different BiCC blocks must pass through these APs. This property holds unconditionally, regardless of algorithm type. Therefore, we can exploit it to decompose execution. That is, each BiCC block can be processed as an independent task, enabling task parallelism across blocks without inter-task synchronization. The BC tree organizes blocks and APs hierarchically, exposing the entry point of each block and making the task decomposition explicit.

V. Structure-Aware Incremental Maintenance

A. Structural Classification of Edge Updates

As the graph dynamically inserts or deletes vertices or edges, this may affect GIR layers in different ways. However, we observe that many updates actually leave the structural connectivity entirely unchanged. For example, an edge inserted within a WCC would not alter that WCC at all.

To leverage this opportunity, we design a *structure-aware incremental maintenance method to identify exactly which regions are affected and repair only the minimum necessary ones*. Particularly, GIR's three-layer hierarchy transforms the maintenance problem. Instead of treating every edge change as a potential graph-wide event, *we classify each update into one of five distinct cases using only pre-maintained metadata*. Each case identifies exactly which layers are affected and confines the repair to the smallest possible region. We call this strategy *Scope-Prune-Update (SPU)* for single-edge updates and extend it to *Filter-Merge-Update (FMU)* for batches.

We primarily focus on *single edge update* (insertion or deletion), as vertex insertions without edges do not affect connectivity and vertex deletions reduce to bulk edge deletions. The central challenge is determining which GIR layers each operation affects. Because GIR stores component labels, SCC membership, DAG ranks, and articulation point metadata for every vertex, it resolves this question through a metadata lookup, not a graph traversal. This classification is only possible as GIR maintains all three structural layers simultaneously.

TABLE I
Scope rules and SPU time complexity per case. $|V_a|$ and $|E_a|$ denote the number of vertices and edges in the affected region; Δr denotes the rank interval for Case 2.

Case	Endpoint relation	Op	Layers	Complexity
1	Different WCCs	Ins	WCC	$\mathcal{O}(1)$
2	Same WCC, diff. SCCs	Ins	SCC (no cycle) SCC+BiCC	$\mathcal{O}(1)$ $\mathcal{O}(\Delta r+\|V_a\|+\|E_a\|)$
3	Same SCC, diff. BiCCs	Ins	BiCC	$\mathcal{O}(\|V_a\|+\|E_a\|)$
4	Same BiCC (no-op)	Ins	BiCC	$\mathcal{O}(1)$
5a	Non-bridge	Del	BiCC	$\mathcal{O}(\|V_a\|+\|E_a\|)$
5b	Bridge within SCC	Del	SCC+BiCC	$\mathcal{O}(\|V_a\|+\|E_a\|)$
5c	Bridge in DAG	Del	WCC+BiCC	$\mathcal{O}(\|V_a\|+\|E_a\|)$

Every edge update $(s \to t)$ falls into one of five cases shown in Figure 4. Particularly, c1 s and t reside in *different WCCs* (insertion); c2 s and t reside in the *same WCC but different SCCs* (insertion); c3 s and t reside in the *same SCC but different BiCCs* (insertion); c4 s and t reside in the *same BiCC* (insertion); c5 edge *deletion*, whose scope depends on bridge status. In addition, Table I summarizes the scope rules and time complexity. For each case, we identify which GIR layers are affected (*scope*), how GIR restricts computation to the smallest subgraph (*prune*), and what structural repair GIR performs (*update*).

Case 1 (Insertion): s and t reside in different WCCs. The inserted edge connects two previously disconnected regions. No directed cycle can form across WCC boundaries, and thus no biconnected structure inside either component changes.

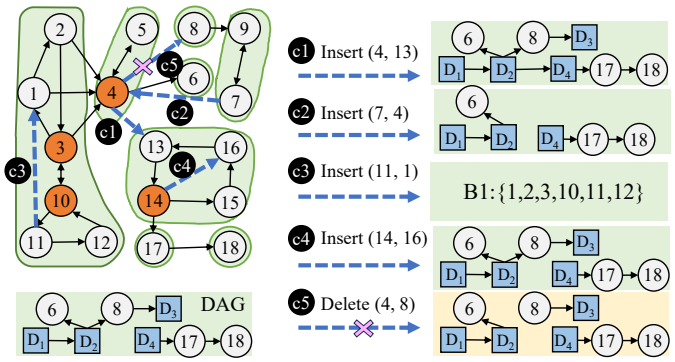
Scope. GIR identifies WCC layer as the only affected layer.

Prune and Update. GIR merges the two components with $O(1)$ time complexity as it only requires a single WCC label update without affecting SCCs or the BC trees.

Example. Inserting an edge $(4, 13)$ (c1 in Figure 4) bridges $\text{WCC}_1 = \{1, ..., 12\}$ and $\text{WCC}_2 = \{13, ..., 18\}$. Thus, GIR connects the two condensation DAGs into one.

Case 2 (Insertion): s and t are in the same WCC but different SCCs. The inserted edge $s \to t$ introduces a new edge between two SCC supernodes in the condensation DAG. The critical question is *whether this edge creates a directed cycle, which would cause two or more SCCs to merge*.

Scope. To answer this without traversing the entire DAG, we design a rank-based method. That is, we assign each SCC a rank, which is a compact topological index maintained incrementally alongside the DAG. If $\text{rank}(\text{SCC}(s)) \leq$



(a) Example graph and DAG (b) Updated graph and scope

Fig. 4. Five cases (c1-c5) for a single edge update. Affected DAG nodes are highlighted in blue, and green shade indicates a WCC.

$\mathrm{rank}(\mathrm{SCC}(t))$, the edge either follows the existing DAG order or connects two SCCs at the same rank with no directed path between them. Neither will form a cycle, and the scope is limited to inserting a new DAG edge with no SCC-level structural change. If $\mathrm{rank}(\mathrm{SCC}(s)) > \mathrm{rank}(\mathrm{SCC}(t))$, the edge goes against the DAG order and may introduce a cycle. The scope then expands to the SCC and BiCC layers.

Prune. GIR restricts cycle detection to the rank interval $[\mathrm{rank}(\mathrm{SCC}(t)), \mathrm{rank}(\mathrm{SCC}(s))]$ in the DAG. This avoids a global DAG search.

Update. If a cycle is detected within this interval, it merges all SCCs on this cycle, updates the condensation DAG locally, and reconstructs the BC trees of the affected SCCs. The time complexity is $\mathcal{O}(\Delta r + |V_a| + |E_a|)$, where $|V_a|$ and $|E_a|$ denote vertex and edge count of the affected region, and Δr denotes the rank interval. If no cycle forms, the cost is $\mathcal{O}(1)$.

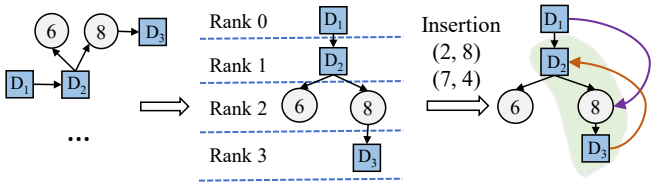
Example. As shown in Figure 5, inserting edge $(7, 4)$ (c2) has $\mathrm{rank}(D_2) < \mathrm{rank}(D_3)$. As $D_2 \to D_3$ is in the original DAG, the edge goes against the DAG order and triggers cycle detection over $[\mathrm{rank}(D_1), \mathrm{rank}(D_2)]$. A cycle is found, and D_2 and D_3 will be merged. The BC tree of the combined SCC will be updated, as shown in Figure 4 (c2). In contrast, inserting $(2, 8)$ satisfies $\mathrm{rank}(\mathrm{SCC}(2)) \leq \mathrm{rank}(\mathrm{SCC}(8))$, thus no cycle can form and only a new DAG edge is added.

Case 3 (Insertion): s and t are in the same SCC but different BiCCs. Both endpoints lie within the same SCC, so no WCC or SCC needs to be changed.

Scope. GIR confines the scope to the BiCC layer. Specifically, only the two BiCC blocks containing s and t are potentially affected.

Prune and Update. The new edge may merge the two blocks into a single larger BiCC, eliminating one or more APs. GIR updates the BC tree accordingly. The time complexity is $\mathcal{O}(|V_a| + |E_a|)$, confined to the two affected BiCC blocks and the local BC tree repair.

Example. Inserting an edge $(11, 1)$ (c3 in Figure 4) will lead to the merge of two blocks. Particularly, vertex 11 is in block $B_1 = \{10, 11, 12\}$ and vertex 1 is in block $B_2 = \{1, 2, 3\}$, both within D_1. Therefore, GIR merges the two blocks into $B_1 = \{1, 2, 3, 10, 11, 12\}$.



(a) Example graph (b) Node rank (c) Scope-prune-update

Fig. 5. Rank-interval pruning for Case c2 (inter-SCC insertion). Inserting edge (2, 8) will not affect SCC, while edge (7, 4) will merge multiple SCCs.

Case 4 (Insertion): s and t are in same BiCC. As both vertices already belong to the same BICC, adding an edge within an already 2-edge-connected subgraph will not create new APs, split any BiCC, or alter SCC or WCC structure.

Scope. GIR identifies the update as a structural no operation for all layers. The time complexity is $\mathcal{O}(1)$.

Example. Inserting an edge $(14, 16)$ (c4 in Figure 4) will not change the GIR as both vertices lie in the same block D_4.

Case 5 (Deletion): The scope of deleting an edge (s, t) depends on whether the edge is a bridge, which is defined as the only edge between two blocks. For this case, GIR further decomposes it into three sub-cases.

Case 5a (non-bridge): If parallel edges or alternative undirected paths exist between s and t, the deletion cannot change WCC, SCC, or higher-level connectivity. GIR confines the scope to the BiCC layer, checks whether any articulation points have changed, and repairs the BC tree locally within the enclosing SCC. The time complexity is $\mathcal{O}(|V_a| + |E_a|)$, confined to the affected BiCC block within the enclosing SCC.

Case 5b (bridge within an SCC): If the edge is a bridge and both endpoints share the same SCC, GIR expands the scope to the SCC layer. It checks whether the directed SCC breaks apart and, if so, partitions the affected SCC, rebuilds the relevant fragment of the condensation DAG, and reconstructs the BC trees of the resulting components. The time complexity is $\mathcal{O}(|V_a| + |E_a|)$, covering the SCC partition check, condensation DAG fragment update, and BC tree reconstruction.

Case 5c (bridge in the condensation DAG): If the deleted edge is an inter-SCC edge and serves as the bridge connecting two regions of the WCC, its removal disconnects the WCC entirely. In this situtation, both WCC and BiCC layers are affected. It will split the WCC into two independent components, partition the condensation DAG accordingly, and update the BC trees on each side to reflect the new component boundaries. The time complexity is $\mathcal{O}(|V_a| + |E_a|)$, covering the WCC split, condensation DAG partition, and BC tree updates on both resulting components.

Example. The edge $4 \to 8$ (c5 in Figure 4) is the only edge between the $\{D_1, D_2\}$ side and the $\{6, 8, D_3\}$ side. Deleting it will split the WCC into two, which will lead to WCC relabeling and BC tree updating on each resulting component.

B. Filter-Merge-Update for Batch Updates

Applying the single edge update strategy (SPU) independently to each update in a batch is correct but inefficient. There are two problems. (i) Some updates cancel or are redundant. That is, if an edge is inserted and later deleted within the same

batch, neither needs to be processed. (ii) Multiple updates may jointly affect the same structural region. Processing them one at a time triggers repeated maintenance on the same region.

To address both problems, we design a filter-merge-update (FMU) strategy by preprocessing the batch before applying SPU logic. Specifically, it involves three stages.

(i) Filter stage removes updates that does not require processing. Particular, we identify two such types of updates. The first is canceling pair. That is, if an edge is inserted and later deleted within the same batch (or vice versa), the two operations cancel and GIR discards both. The second is structural no operations. Particularly, based on the five-case classification in Table I, we remove any Case 4 updates (i.e., s and t are in the same BiCC), as they will not affect any GIR layer. This filter step reduces both the number of updates and the total maintenance work.

(ii) Merge will group the remaining updates by the structural region they may jointly affect. We first group the updates whose endpoints share the same WCC. Within each WCC group, we identify updates whose SCC endpoints have overlapping DAG rank intervals and merges them into a single SCC-level maintenance region. Further, we group updates that touch the same local articulation structure at the BiCC level. This grouping has two benefits. First, it amortizes repeated maintenance. That is, the updates that would trigger repairs on the same region are handled together in a single pass. Second, it identifies structurally independent groups, each of which becomes a natural unit for parallel execution.

(iii) Update. Finally, GIR applies the single edge update logic to each merged region rather than to individual edges. A set of inserted edges may jointly create a cycle and merge several SCCs, even though none of the edges alone would trigger a merge. GIR detects such structural transitions over the merged region and repairs the corresponding WCC, SCC, and BC tree structures in a coordinated way. Once the batch is complete, GIR processes independent merged regions concurrently. This makes FMU a parallel and amortized extension of SPU for high-throughput update streams.

VI. GIR-Enabled Two-Level Parallel Execution

A. The Two-Level Parallel Execution Model

Most parallel graph computation frameworks rely on *data parallelism* [8]–[10]. That is, they process one task with many vertices or edges simultaneously using all the available threads (or cores). This approach leaves a second, orthogonal source of parallelism unexploited, i.e., *task parallelism*, where structurally independent subgraphs execute as concurrent tasks. It is difficult to achieve on regular graphs because structural boundaries are not stored and must be recomputed from scratch at query time.

Fortunately, GIR eliminates this barrier by persistently maintaining the graph's three-level structural decomposition. Motivated by this opportunity, we design a *two-level parallelism model, i.e., task parallelism across blocks, and data parallelism within each block*. In the following, we first discuss the theoretical foundation for this model and then apply it to BFS and other algorithms.

Let $G = (V, E)$ be a directed graph. GIR maintains a three-level structural hierarchy. At the WCC level, GIR stores $\{W_1, \ldots, W_p\}$. At the SCC level, GIR stores $\{C_1, \ldots, C_q\}$ and their condensation DAG $\mathcal{D} = (\mathcal{V}_\mathcal{D}, \mathcal{E}_\mathcal{D})$. At the BiCC level, for each SCC C_k, GIR stores the BiCC blocks $\{B_1^k, \ldots, B_{r_k}^k\}$ of the underlying undirected graph $\hat{G}[C_k]$, organized as a BC tree $\mathcal{T}_k$. A vertex a is an *articulation point* of C_k if its removal from $\hat{G}[C_k]$ increases the number of connected components. We write ∂B for the set of articulation points on the boundary of block B. We write $G[B]$ for the subgraph of G induced by the vertices of B.

These three levels provide three distinct structural boundaries at which computation can be partitioned. We formalize each with a separation lemma.

Lemma 1 (WCC separation)**.** *Let W_i and W_j be two distinct WCCs of G. No path exists between any vertex in W_i and any vertex in W_j. Any algorithm whose output for a vertex v depends only on vertices reachable from v is fully independent across WCC boundaries.*

Lemma 2 (SCC-DAG separation)**.** *Let C_i and C_j be two SCCs such that C_j is not reachable from C_i in the condensation DAG $\mathcal{D}$. No directed path exists from any vertex in C_i to any vertex in C_j. Any algorithm whose output for vertices in C_i depends only on vertices reachable from C_i is independent of C_j.*

Lemma 3 (AP separation)**.** *Let B_i and B_j be two distinct BiCC blocks within the same SCC C_k, with $u \in B_i$ and $v \in B_j$. In graph $\hat{G}[C_k]$, every path between u and v passes through every AP on the unique path between B_i and B_j in the BC tree $\mathcal{T}_k$. Consequently, the distance $d(u, v) = d(u, a^*) + d(a^*, v)$ for any AP a^* on the BC tree path from B_i to B_j. Since every directed edge in $G[C_k]$ is also an undirected edge in $\hat{G}[C_k]$, every directed path between u and v is also an undirected walk, and therefore passes through every AP on the BC tree path. Hence the distance decomposition extends to directed shortest-path distances as well.*

Proof. Lemmas 1 and 2 follow directly from the definitions of WCCs and SCCs. Lemma 3 follows from the definition of biconnected components. □

Together, these three lemmas establish that GIR's three-level hierarchy provides three provably independent partitions of the graph. We now define which structural level is the right partition boundary for a given algorithm.

Definition 1 (Dependency level)**.** *The dependency level of an algorithm $\mathcal{A}$, written $\mathcal{L}(\mathcal{A}) \in \{WCC, SCC, BiCC\}$, is the coarsest level at which $\mathcal{A}$'s computation on a structural unit S is self-contained given boundary values at ∂S.*

For example, a connected component labeling algorithm has $\mathcal{L}(\mathcal{A}) = WCC$ as each component is fully independent with no boundary values. A reachability query from a source s has $\mathcal{L}(\mathcal{A}) = SCC$ because only SCCs reachable from C_s in $\mathcal{D}$ participate. A shortest path algorithm has $\mathcal{L}(\mathcal{A}) = BiCC$

because shortest paths between blocks decompose exactly at APs based on Lemma 3.

Definition 2 (Boundary-decomposable algorithm)**.** *An algorithm $\mathcal{A}$ with dependency level $\mathcal{L}(\mathcal{A})$ is boundary-decomposable if it satisfies two conditions.*

- ***Local decomposability.*** *For each structural unit S at level $\mathcal{L}(\mathcal{A})$, there exists a function f_S such that $\mathcal{A}(v) = f_S\big(v,\ \lambda(\partial S),\ G[S]\big)$ for all $v \in S$, where $\lambda(a)$ is the externally provided boundary value at $a \in \partial S$.*
- ***Boundary composability.*** *The global output of $\mathcal{A}$ can be assembled by propagating λ values across structural boundaries in a single pass.*

Local decomposability means the computation inside unit S requires no information from outside S beyond the values at its boundary. *Boundary composability* means the global result follows from combining local results at boundaries without a second global pass. We now state the main result.

Theorem 1 (Two-level parallel execution)**.** *Let $\mathcal{A}$ be a boundary-decomposable algorithm with dependency level $\mathcal{L}(\mathcal{A})$. GIR executes $\mathcal{A}$ correctly in two levels:*

- ***Level 1 (structural partitioning).*** *GIR identifies all structural units $\{S_1, \ldots, S_k\}$ at level $\mathcal{L}(\mathcal{A})$ that are relevant to the query, using the separation lemma at that level. Irrelevant units are discarded without affecting correctness.*
- ***Level 2 (independent task execution).*** *GIR assigns each relevant unit S_i as one independent task and executes all k tasks concurrently. For each task, GIR provides boundary values $\lambda(\partial S_i)$ before execution. After all tasks complete, GIR assembles global result via boundary composability.*

The parallel execution time of Level 2 is $\mathcal{O}(\max_i T(S_i))$, where $T(S_i)$ is the sequential execution time of $\mathcal{A}$ on unit S_i. This compares to $\mathcal{O}(\sum_i T(S_i))$ for sequential execution. The task parallel speedup scales with k, the number of relevant structural units.

Proof. Correctness. By the separation lemma at level $\mathcal{L}(\mathcal{A})$, all information that crosses boundary of unit S_i passes through ∂S_i. Before Level 2 begins, GIR computes $\lambda(\partial S_i)$ for all i via the structural traversal in Level 1. By local decomposability, each task S_i computes its correct local output using only $G[S_i]$ and $\lambda(\partial S_i)$. By boundary composability, combining local outputs at boundaries produces the globally correct result.

Independence. WCCs and SCCs each partition V, so tasks at these levels share no vertices. At the BiCC level, blocks partition the *edges* of G, not the vertices. That is, APs are shared across adjacent blocks, but they appear only as read-only boundary values $\lambda(\partial S_i)$ that are fixed before Level 2 begins. All non-boundary internal vertices belong to exactly one block. Since no two tasks share any internal vertex or write to any shared state, their executions do not interfere. Tasks require no synchronization during Level 2.

Parallel time bound. Since all k tasks are independent and execute concurrently, the parallel execution time is determined by the slowest task. The speedup of this execution model over a regular graph method grows with k. □

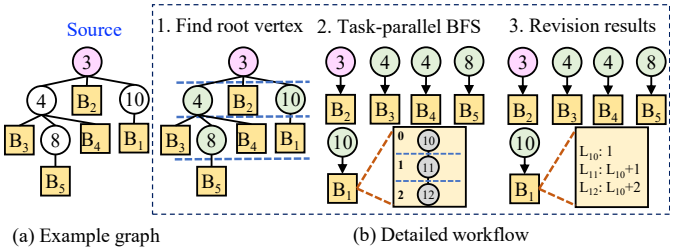


Fig. 6. Concurrent BFS on GIR from root vertex 3. (a) shows an example graph in BC tree. (b) shows the BFS computation workflow.

B. Applications of the Two-Level Parallel Execution Model

We demonstrate the two-level parallel execution model on three algorithm families, i.e., graph traversal, path finding, and centrality. For each, we identify its dependency level per Definition 1, verify that it is boundary-decomposable per Definition 2, and describe how Theorem 1 applies. We use BFS as the primary concrete example.

BFS and graph traversal. BFS is applied hierarchically across two structural levels. At the SCC level ($\mathcal{L} = SCC$), the output for any vertex v depends only on vertices reachable from the source s in G, and reachability is SCC-bounded by Lemma 2, while unreachable SCCs are pruned entirely. Within each retained SCC, computation further decomposes at the BiCC level ($\mathcal{L} = BiCC$). By Lemma 3, the BFS level of any vertex $v \in B$ equals to the entry level $\lambda(a^*)$ at B's entry AP a^* plus the local BFS level of v within B. This satisfies local decomposability with $f_B(v, \lambda(\partial B), G[B]) = \lambda(a^*) + \ell_{\text{local}}(v, B)$. Global BFS levels assemble by propagating entry levels across the BC tree, satisfying boundary composability. BFS is thus boundary-decomposable and Theorem 1 applies.

Given a source s, GIR executes BFS in four concrete stages.

Stage 1: SCC pruning via the condensation DAG. GIR traverses $\mathcal{D}$ from the SCC C_s containing s and retains only the reachable set $\mathcal{R} = \{C \in \mathcal{V}_\mathcal{D} : C_s \rightsquigarrow C\}$. All SCCs outside $\mathcal{R}$ are provably irrelevant by Lemma 2. The time complexity of this stage is $\mathcal{O}(|\mathcal{V}_\mathcal{D}| + |\mathcal{E}_\mathcal{D}|)$.

Stage 2: Task decomposition via the BC tree. Within each retained SCC, GIR traverses the BC tree $\mathcal{T}_k$ in BFS order starting from the block B_s containing s, to identify the set of reachable blocks $\mathcal{B}_s$. For the example graph in Figure 6(a), starting form root vertex 3, B_1-B_5 are identified as reachable. During this traversal, GIR computes the entry BFS level $\lambda(a_i^*)$ for each block $B_i \in \mathcal{B}_s$ in topological order along $\mathcal{T}_k$. That is, the entry level of B_s itself is 0, and the entry level of each subsequent block is determined from its parent's entry AP. Particularly, for the example graph, the entry levels of B_1, B_2, B_3, B_4, and B_5 are 1, 0, 1, 1, 2, respectively. This sequential pre-computation has time complexity $\mathcal{O}(|\mathcal{B}_s|)$. Each block then becomes one independent task with all boundary values fixed. By Lemma 3, blocks share no internal vertices and communicate only through APs. Thus, all tasks in $\mathcal{B}_s$ are mutually independent once Stage 2 completes.

Stage 3: Concurrent BFS within each block. GIR executes all $|\mathcal{B}_s|$ tasks concurrently. Each task runs BFS from its entry AP a_i^*, computing local BFS levels $\ell_{\text{local}}(v, B_i)$ for all $v \in B_i$.

TABLE II
CHARACTERISTICS OF THE GRAPH DATASETS.

Graph	Abbr.	# Nodes	# Edges	Avg Deg.
twitter_combined	TC	81.3K	2.4M	59.6
web-Google	WG	876K	5.1M	11.7
wiki-topcats	WT	1.8M	28.5M	31.8
cit-Patents	CP	3.8M	16.5M	8.8
soc-LiveJournal	LJ	4.9M	69M	28.5
twitter	TW	61.6M	2.4B	35.3
friendster	FS	65.6M	1.8B	44.6

Tasks require no synchronization during execution and are assigned to independent CPU worker threads, with all threads running simultaneously. The parallel time complexity of this stage is $\mathcal{O}(\max_{B_i \in \mathcal{B}_s}(|V_{B_i}| + |E_{B_i}|))$. For the five blocks shown in Figure 6(b), they can run simutaneously.

Stage 4: Global level revision. GIR revises the global BFS levels via boundary composability. For each vertex $v \in B_i$, the global level is $\ell(v) = \lambda(a_i^*) + \ell_{\text{local}}(v, B_i)$. This merge is embarrassingly parallel and requires no further traversal. For block B_1 in Figure 6(b), the global BFS level of each vertex becomes the sum of its local level plus 10's level.

Path finding. SSSP and k-shortest paths (KSP) have dependency level $\mathcal{L} = \textit{BiCC}$. By Lemma 3, every path between $u \in B_i$ and $v \in B_j$ passes through the AP a^* between them. Therefore $d(s, v) = d(s, a^*) + d_{\text{local}}(a^*, v, B_j)$, where d_{local} is the shortest-path distance within block B_j. This satisfies local decomposability with boundary value $\lambda(a^*) = d(s, a^*)$. Global distances assemble by propagating λ values across the BC tree, satisfying boundary composability. GIR applies Level 1 to identify blocks $\mathcal{B}_s$ on any source-to-target path, then executes independent shortest-path tasks on each block concurrently in Level 2. Distances are merged at APs to produce globally correct results.

Centrality. Betweenness centrality and closeness centrality repeatedly solve shortest-path subproblems and have dependency level $\mathcal{L} = \textit{BiCC}$ under the AP/BC tree reduction [29]. APs act as mandatory waypoints on shortest paths by Lemma 3, so any shortest path that crosses a block boundary must pass through an AP. GIR eliminates cross-boundary redundancy exactly at APs, reducing the number of shortest-path computations from $\mathcal{O}(|V|^2)$ toward $\mathcal{O}(|V_{B_i}|^2)$ per block. GIR then assigns one centrality task per block and executes all tasks concurrently, assembling global centrality scores at APs via boundary composability.

VII. EXPERIMENT

A. Experimental Setup

We run all the experiments on a dual-socket server with two AMD EPYC 9334 32-core CPUs, up to 64 threads in total. We compile all implementations with GCC 12.3 with optimization level `-O3`, and use OpenMP 5.1.

Dataset. We test seven large-scale real-world graph datasets. Table II summarizes the details. All downloaded from the Stanford Large Network Dataset Collection (SNAP) [30]. They span citation networks (CP), social networks (LJ, TC, TW, and FS), and web graphs (WG and WT). Among them, there are two huge graphs with over 1 billion edges.

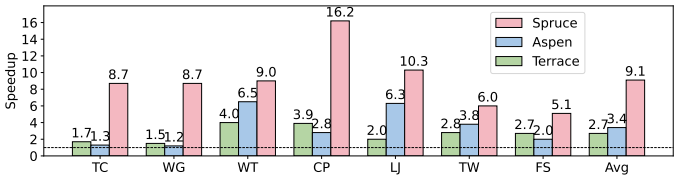


Fig. 7. Insertion-only speedup of GIR over Terrace, Aspen, and Spruce.

Algorithm. We test three representative graph algorithms, including breadth-first search (BFS), single-source shortest paths (SSSP), and betweenness centrality (BC). We do not test WCC, SCC, or BiCC as they are already in GIR.

Baseline. We compare GIR against three recent dynamic graph computation systems, i.e., Terrace [18], Aspen [20], and Spruce [19]. Particular, (i) *Terrace* targets skewed graphs and uses a hierarchical container that adapts the adjacency representation to vertex degree, balancing locality, update cost, and concurrency. (ii) *Aspen* targets low-latency graph streaming by maintaining compressed purely functional trees and exposing an immutable snapshot interface for analytics, enabling many concurrent readers and typically a single writer that publishes new versions. (iii) *Spruce* is a space-efficient dynamic graph representation that combines compact indexing with update-friendly adjacency organization and supports concurrent reads and writes through a read-optimized protocol.

B. Update Throughput Comparison

We evaluate *update throughput* under three settings, i.e., insertion-only, deletion-only, and mixed. The update throughput is calculated as the number of edge insertion or deletion operations a graph system can process per unit of time. Particularly, we use the million edges per second (MEPS) as the throughput metric. For each graph, we generate randomized insertion and deletion operations to emulate evolving graph streams as previous works have done.

Insertion-only. Figure 7 reports the insertion-only speedup of GIR over compared methods.For each graph, we randomly generate $|E|$ edge insertions on top of the existing graph and measure average throughput. (i) GIR consistently outperforms all the compared methods on every dataset. Particularly, GIR achieves an average of $9.1\times$, $3.4\times$, and $2.7\times$ over Spruce, Aspen, and Terrace, respectively.

(ii) The largest gains appear on large, structurally irregular graphs. For example, on the CP graph, GIR achieves $16.2\times$ over Spruce, $3.9\times$ over Terrace, and $2.8\times$ over Aspen. Similarly, on the LJ graph, GIR achieves $10.3\times$, $2\times$, and $6.3\times$ over Spruce, Terrace, and Aspen, respectively. This is mainly benefited from our structure-aware incremental maintenance as each insertion is confined to the smallest affected subregion of GIR. Therefore, the repair cost scales with the local structural change rather than total graph size.

(iii) Performance gains remain stable on smaller graphs. On TC and WG graphs, GIR achieves $8.7\times$ over Spruce and 1.3-$1.5\times$ over Aspen and Terrace. Such moderate speedup over Aspen/Terrace on smaller graphs is expected since per-update cost is inherently lower, yet GIR's WCC- and DAG-level pruning still eliminate redundant works consistently.

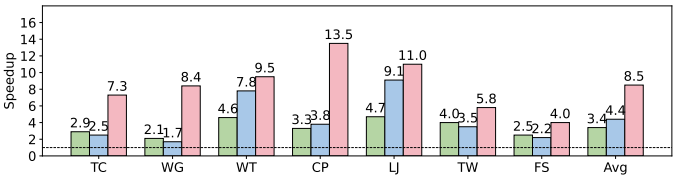


Fig. 8. Deletion-only speedup of GIR over Terrace, Aspen, and Spruce.

TABLE III
MIXED UPDATE THROUGHPUT (MEPS ↑). THE BEST PERFORMANCE OF EACH GRAPH IS HIGHLIGHTED. THE SPEEDUP OF GIR OVER CURRENT BEST IS IN BOTTOM. LAST COLUMN DENOTES AVERAGE SPEEDUP.

	TC	WG	WT	CP	LJ	TW	FS	Speedup
Spruce	3.5	3	3	3.1	2.7	2.9	2.5	*8.5×*
Aspen	20.2	18.9	4.3	31.2	4.6	8.9	10	*1.8×*
Terrace	17.5	16.4	6.7	31.2	8.1	7.6	7.8	*1.9×*
GIR	26.3	24.6	27.6	43.7	25.9	16	14	
	1.3×	*1.3×*	*4.1×*	*1.4×*	*3.2×*	*1.8×*	*1.4×*	

Deletion-only. Figure 8 reports the deletion-only speedup of GIR over compared methods. For each graph, we randomly generate $|E|$ edge deletions on the existing graph and measure average throughput. Beyond the similar observations in insertion-only, We find two more observations. *(i)* The speedups of deletions are higher than insertions over Aspen and Terrace. On average, the speedup over Aspen grows from $3.4\times$ to $4.4\times$, and over Terrace from $2.7\times$ to $3.4\times$. These are because Aspen needs to rebuild functional tree paths and Terrace needs to restructure hierarchical adjacency upon deletions, which are intrinsically more expensive than insertions. Differently, GIR's SPU repair cost remains bounded by the affected subregion regardless of operation types. *(ii)* The speedup over Spruce is slightly lower for deletion than insertion. Particularly, the average speedup drops from $9.1\times$ to $8.5\times$, and on CP from $16.2\times$ to $13.5\times$. Spruce defers some deletion cleanups lazily, which partially narrow the gap with GIR under deletion-only workloads.

Mixed insertion and deletion. Table III reports throughput under the mixed workload. We perform $10 \times |E|$ operations in total, with 50% insertions and 50% deletions interleaved uniformly at random on the existing graph. The speedup of GIR over Aspen and Terrace is lower in mixed mode ($1.8\times$ and $1.9\times$) than insertion-only ($3.4\times$, $2.7\times$) or deletion-only ($4.4\times$, $3.4\times$). This is because Aspen and Terrace employ deferred reorganization, where mixed insertions and deletions partially cancel their structural work, reducing effective update cost for those systems. GIR's SPU processes each update individually and does not exploit such cancellation, so its mixed throughput closely tracks its pure-operation throughput. Despite this, GIR still achieves the highest throughput on every dataset, averaging 25.4 MEPS versus 14 (Aspen), 13.6 (Terrace), and 3 (Spruce).

C. Graph Analytics Performance

GIR is evaluated on BFS, SSSP, and BC. Table IV reports the speedup of GIR over each baseline across all datasets. Note, TW and FS are tested with 32 threads, since the compared methods encounter out-of-memory errors at 64 threads. All the other graphs are evaluated with 64 threads.

TABLE IV
SPEEDUPS (×) OF GIR OVER COMPARED METHODS. TW AND FS ARE TESTED WITH 32 THREADS, SINCE THE COMPARED METHODS ENCOUNTER OUT-OF-MEMORY ERRORS AT 64 THREADS.

Alg.	Method	TC	WG	WT	CP	LJ	TW*	FS*	Avg
BFS	Terrace	1.5	1.2	1.2	1.8	1.5	1.1	1.1	1.3
	Aspen	1.1	1.2	1.4	1.4	1.2	1.2	1.6	1.3
	Spruce	3	2.7	1.4	8.6	1.6	1.8	1.9	3
SSSP	Spruce	7	2.4	3.1	1.9	1.9	1.1	1.4	2.7
BC	Terrace	2.1	1.8	1.9	1.5	1.3	1.5	1.5	1.7
	Aspen	2	2.1	2.1	1.6	2.1	1.2	1.1	1.7

BFS is evaluated against all three baselines. We make three observations. *(i)* GIR consistently outperforms all baselines on every dataset. On average, GIR achieves $3\times$ over Spruce, $1.3\times$ over Terrace, and $1.3\times$ over Aspen. *(ii)* The largest gains appear on CP and TC. For the CP graph, GIR achieves $8.6\times$ over Spruce, the highest single-dataset BFS speedup, because the GIR hierarchy exposes many small independent BiCC blocks and converts BFS into many short parallel traversal tasks. For the TC graph, GIR achieves $3\times$ over Spruce for the same reason. Against Terrace and Aspen, the speedups are more moderate (1.1 to $1.8\times$), driven by WCC- and DAG-level pruning that eliminate unreachable components before any edge is examined. *(iii)* The performance gains are smaller on TW and FS. These graphs are dominated by a few very large SCCs. Thus, the number of independent BiCC blocks is limited and intra-block traversal dominates the runtime, reducing the benefit of block-level task parallelism.

SSSP is evaluated against Spruce only, as Terrace and Aspen do not support weighted graphs. We also make three observations. *(i)* GIR outperforms Spruce on SSSP across all seven datasets, averaging $2.7\times$. *(ii)* The largest gain is on TC graph ($7\times$), where the DAG prunes a large fraction of unreachable SCCs before any relaxation step, substantially reducing the effective graph size. For the WT ($3.1\times$) and WG ($2.4\times$) graphs, the DAG-guided topological order lets GIR process SCCs level by level without revisiting settled regions. *(iii)* The performance gains are moderate on LJ, CP, and FS graphs (1.4 to $1.9\times$). On these graphs, the reachable SCC subgraph covers a larger fraction of the total graph. Thus, the DAG pruning is less aggressive, while BiCC block decomposition still reduces intra-component relaxation overhead.

BC is evaluated against Terrace and Aspen, while Spruce does not provide a BC implementation. We make two observations. *(i)* GIR consistently outperforms both baselines, averaging $1.7\times$ over both Terrace and Aspen. *(ii)* The highest performance gains appear on TC and WG. For the TC graph, GIR achieves $2.1\times$ over Terrace and $2\times$ over Aspen. For the WG graph, GIR achieves $1.8\times$ and $2.1\times$ over Terrace and Aspen, respectively. Since BC issues an SSSP from every vertex, GIR's BiCC block decomposition is especially effective. That is, many small independent blocks allow GIR to distribute per-source work across many concurrent tasks, substantially shortening the critical path.

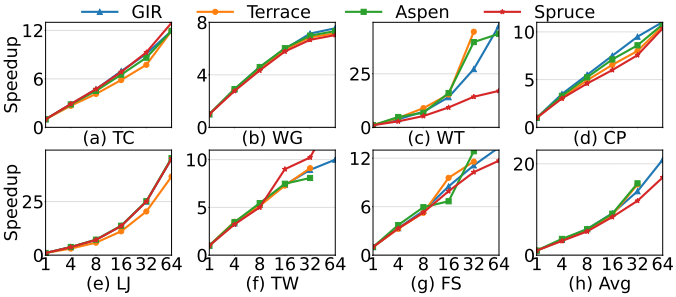


Fig. 9. The BFS scalability against thread number. For TW and FS, Terrace and Aspen fail at 64 threads. For WT, Terrace fails at 64 threads.

D. Scalability

We test the scalability of GIR by varying the number of threads from 1 to 64 across all seven datasets. We take the BFS as a representative example. Figure 9 shows the speedup of each method normalized to its own single-thread runtime. Note that Terrace and Aspen fail at 64 threads on TW and FS, and Terrace fails at 64 threads on WT.

Particularly, we make three observations. *(i)* On average, GIR achieves $20.9\times$ speedups with 64 threads, way higher than Spruce's $17\times$. Terrace and Aspen achieve $15.5\times$ and $15.7\times$ with 32 threads, respectively. They are omitted at 64 threads due to failures on TW and FS.

(ii) GIR achieves the highest scaling on graphs with many independent BiCC blocks. On WT, GIR achieves $47.4\times$ at 64 threads, higher than Aspen ($43.6\times$) and Spruce ($17\times$). On LJ, all methods scale similarly well, with GIR reaching $45\times$ at 64 threads. On TC, GIR achieves $11.9\times$ at 64 threads, mainly due to TC's exceptionally high count of small BiCC blocks.

(iii) Scaling is more modest on graphs dominated by a few large SCCs. On WG and CP, all methods scale similarly, with GIR reaching $7.5\times$ and $11\times$ at 64 threads. On TW and FS, GIR achieves $8.9\times$ and $11.1\times$ at 32 threads before Terrace and Aspen fail to scale further.

VIII. Related Work

Dynamic graph computation. Besides the extensively discussed Terrace [18], Spruce [19], and Aspen [20], Stinger [14] aggregates edge lists into fixed-size blocks with a hash-table index for point queries. GraphOne [31] combines an adjacency list with an edge log that is periodically merged in batches. Sortledton [32] replaces edge lists with an unrolled skip list offering $O(\log |E|)$ update and lookup with transactional support. Teseo [33] uses packed memory arrays as leaves of B+ trees with optimistic locking for concurrent access. CuckooGraph [17] proposes a hash-array-based structure that adaptively resizes through transformation and denylist techniques. Other works include LiveGraph [34], and LLAMA [35].

Query-dependent incremental computation. A separate line of work attempts to reuse prior computational results across updates [12], [36]. *Khanda et al.* [21] maintain an SSSP tree rooted at a fixed source after each update, recomputing only the affected subtree. *Kickstarter et al.* [37] retain approximate monotonic results and propagates corrections for invalidated values. *Fan et al.* [38] propose a general framework for incrementalizing arbitrary graph algorithms by detecting affected regions across updates. *Bauer and Wagner* [39] study batch dynamic SSSP experimentally, showing reuse is only effective when the affected subgraph is small.

Intermediate representations (IRs), such as LLVM IR [22], decouple source languages from back-end optimization passes, enabling reusable analysis across many compilation stages. Several graph domain-specific languages adopt a related algorithm-execution separation. GraphIt [10] separates algorithm descriptions from scheduling decisions. Galois [40] decouples operator logic from parallel execution policies. Ligra [8], Green-Marl [41], Gemini [42], and Pregel [43] target shared- and distributed-memory execution on static snapshots.

IX. Dicussion

Algorithm independence. The three structural layers in GIR do not depend on any specific query source, target, or algorithm. The DAG and BC trees are maintained unconditionally after each graph update. Any downstream algorithm can use these structures without prior knowledge.

Integration with other dynamic graph systems. GIR is designed to operate above existing dynamic graph representation systems, not to replace them. It can be integrated with formats such as PMA [44], DCSR [45], Terrace [18], and DGAP [46]. The storage layer is responsible for maintaining adjacency data efficiently under updates. GIR is responsible for maintaining the structural invariants. This separation improves modularity and allows GIR to be added to an existing system without redesigning the storage stack.

Extensibility. The current instantiation of GIR uses WCC, SCC, and BiCC because they provide a strong balance between expressiveness and computational feasibility at scale. The framework is not limited to these three. Additional structural indices, such as dominator trees or reachability indices, can be incorporated when they provide value for a target workload.

Limitations. GIR is most beneficial for algorithms whose execution can be pruned, decomposed, or reordered using structural boundaries. Algorithms that aggregate globally across all vertices, such as PageRank [47] and label propagation [48], might not directly benefit from the condensation DAG or BC tree. They operate on all edges regardless of source, so structural boundaries do not reduce their work. However, these algorithms still benefit from WCC-level partitioning. That is, separating WCCs can be processed independently and in parallel. After that, the updates confined to one WCC require only that component's scores to be updated. We would not claim of benefit for global aggregation algorithms beyond this.

X. Conclusion

We design GIR, a query-agnostic graph intermediate representation. GIR persistently maintains a three-level structural hierarchy of WCC, SCC, and BiCC, confines maintenance to the minimum affected region, and enables task parallelism. Experiments on seven real-world graphs show that GIR achieves $1.8\times$ more update throughput than state-of-the-art methods. For downstream graph analytics, GIR achieves an average of $1.3\times$, $2.7\times$, and $1.7\times$ speedups for BFS, SSSP, and betweenness centrality, respectively.

References

[1] A. Mislove, M. Marcon, K. P. Gummadi, P. Druschel, and B. Bhattacharjee, "Measurement and analysis of online social networks," in *Proceedings of the 7th ACM SIGCOMM conference on Internet measurement*, 2007, pp. 29–42. [Online]. Available: https://dl.acm.org/doi/10.1145/1298306.1298311

[2] J. Leskovec, J. Kleinberg, and C. Faloutsos, "Graphs over time: densification laws, shrinking diameters and possible explanations," in *Proceedings of the eleventh ACM SIGKDD international conference on Knowledge discovery in data mining*, 2005, pp. 177–187. [Online]. Available: https://dl.acm.org/doi/10.1145/1081870.1081893

[3] R. Kikas, G. Gousios, M. Dumas, and D. Pfahl, "Structure and evolution of package dependency networks," in *2017 IEEE/ACM 14th International Conference on Mining Software Repositories (MSR)*. IEEE, 2017, pp. 102–112. [Online]. Available: https://doi.org/10.1109/MSR.2017.55

[4] C. Y. Lee, "An algorithm for path connections and its applications," *IRE Transactions on Electronic Computers*, vol. EC-10, no. 3, pp. 346–365, 1961. [Online]. Available: https://doi.org/10.1109/TEC.1961.5219222

[5] M. Thorup, "Undirected single source shortest paths in linear time," in *Proceedings 38th Annual Symposium on Foundations of Computer Science*. IEEE, 1997, pp. 12–21. [Online]. Available: https://doi.org/10.1145/316542.316548

[6] R. Seidel, "On the all-pairs-shortest-path problem," in *Proceedings of the twenty-fourth annual ACM symposium on Theory of computing*, 1992, pp. 745–749. [Online]. Available: https://dl.acm.org/doi/10.1145/129712.129784

[7] U. Brandes, "A faster algorithm for betweenness centrality," in *Journal of Mathematical Sociology*, vol. 25, no. 2, 2001, pp. 163–177. [Online]. Available: https://doi.org/10.1080/0022250X.2001.9990249

[8] J. Shun and G. E. Blelloch, "Ligra: A lightweight graph processing framework for shared memory," in *Proc. ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP)*, 2013, pp. 135–146. [Online]. Available: https://doi.org/10.1145/2517327.2442530

[9] Y. Wang, A. Davidson, Y. Pan, Y. Wu, C. Yang, and J. D. Owens, "Gunrock: A high-performance graph processing library on the gpu," in *Proc. ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP)*, 2016, pp. 11:1–11:12. [Online]. Available: https://doi.org/10.1145/2851141.2851145

[10] Y. Zhang, M. Halim, S. Kamil, J. Shun, and S. Amarasinghe, "Graphit: A high-performance graph dsl," in *Proc. ACM on Programming Languages (OOPSLA)*, vol. 2, no. OOPSLA, 2018, pp. 121:1–121:30. [Online]. Available: https://doi.org/10.1145/3276491

[11] K. Hanauer, J. Sun, and F. Meyerhenke, "Fully dynamic graph algorithms: A survey," *ACM Computing Surveys*, vol. 55, no. 9, pp. 1–38, 2022. [Online]. Available: https://doi.org/10.1145/3555806

[12] M. Besta, M. Fischer, V. Kalavri, M. Kapralov, and T. Hoefler, "Practice of streaming processing of dynamic graphs: Concepts, models, and systems," *IEEE Transactions on Parallel and Distributed Systems*, vol. 34, no. 6, pp. 1860–1876, 2021. [Online]. Available: https://doi.org/10.1109/TPDS.2021.3131677

[13] J. Brank and J. Leskovec, "The download estimation task on kdd cup 2003," *ACM SIGKDD Explorations Newsletter*, vol. 5, no. 2, pp. 160–162, 2003. [Online]. Available: https://dl.acm.org/doi/10.1145/980972.980997

[14] D. Ediger, R. McColl, J. Riedy, and D. A. Bader, "Stinger: High performance data structure for streaming graphs," in *2012 IEEE Conference on High Performance Extreme Computing*. IEEE, 2012, pp. 1–5. [Online]. Available: https://doi.org/10.1109/HPEC.2012.6408680

[15] F. Busato, O. Green, N. Bombieri, and D. A. Bader, "Hornet: An efficient data structure for dynamic sparse graphs and matrices on gpus," in *2018 IEEE High Performance extreme Computing Conference (HPEC)*. IEEE, 2018, pp. 1–7. [Online]. Available: https://doi.org/10.1109/HPEC.2018.8547541

[16] Y. Awad, S. Yalamanchili, and V. Prasanna, "Dynamic graph processing on gpus," *IEEE Transactions on Parallel and Distributed Systems (TPDS)*, vol. 31, no. 10, pp. 2368–2381, 2020. [Online]. Available: https://doi.org/10.1109/IPDPS47924.2020.00081

[17] Z. Fan, Y. Cai, Z. Liu, J. Guo, X. Fan, T. Yang, and B. Cui, "Cuckoograph: A scalable and space-time efficient data structure for large-scale dynamic graphs," in *2025 IEEE 41st International Conference on Data Engineering (ICDE)*. IEEE, 2025, pp. 1265–1277. [Online]. Available: https://doi.org/10.1109/ICDE65448.2025.00099

[18] P. Pandey, B. Wheatman, H. Xu, and A. Buluc, "Terrace: A hierarchical graph container for skewed dynamic graphs," in *Proceedings of the 2021 International Conference on Management of Data*, 2021, pp. 1372–1385. [Online]. Available: https://doi.org/10.1145/3448016.3457313

[19] J. Shi, B. Wang, and Y. Xu, "Spruce: a fast yet space-saving structure for dynamic graph storage," *Proceedings of the ACM on Management of Data*, vol. 2, no. 1, pp. 1–26, 2024. [Online]. Available: https://doi.org/10.1145/3639282

[20] L. Dhulipala, G. E. Blelloch, and J. Shun, "Low-latency graph streaming using compressed purely-functional trees," in *Proceedings of the 40th ACM SIGPLAN conference on programming language design and implementation*, 2019, pp. 918–934. [Online]. Available: https://doi.org/10.1145/3314221.3314598

[21] A. Khanda, S. Srinivasan, S. Bhowmick, B. Norris, and S. K. Das, "A parallel algorithm template for updating single-source shortest paths in large-scale dynamic networks," *IEEE Transactions on Parallel and Distributed Systems*, vol. 33, no. 4, pp. 929–940, 2021. [Online]. Available: https://doi.org/10.1109/TPDS.2021.3084096

[22] C. Lattner and V. Adve, "Llvm: A compilation framework for lifelong program analysis & transformation," in *Proc. International Symposium on Code Generation and Optimization (CGO)*, 2004, pp. 75–86. [Online]. Available: https://doi.org/10.1109/CGO.2004.1281665

[23] B. Wheatman and H. Xu, "Packed compressed sparse row: A dynamic graph representation," in *2018 IEEE High Performance extreme Computing Conference (HPEC)*. IEEE, 2018, pp. 1–7. [Online]. Available: https://doi.org/10.1109/HPEC.2018.8547566

[24] A.-L. Barabási and R. Albert, "Emergence of scaling in random networks," *science*, vol. 286, no. 5439, pp. 509–512, 1999. [Online]. Available: https://doi.org/10.1126/science.286.5439.509

[25] Y. S. U. Vishkin and Y. Shiloach, "An o (log n) parallel connectivity algorithm," *J. algorithms*, vol. 3, pp. 57–67, 1982. [Online]. Available: https://doi.org/10.1016/0196-6774(82)90008-6

[26] M. Sharir, "A strong-connectivity algorithm and its applications in data flow analysis," *Computers & Mathematics with Applications*, vol. 7, no. 1, pp. 67–72, 1981. [Online]. Available: https://doi.org/10.1016/0898-1221(81)90008-0

[27] J. Westbrook and R. E. Tarjan, "Maintaining bridge-connected and biconnected components on-line," *Algorithmica*, vol. 7, no. 1, pp. 433–464, 1992. [Online]. Available: https://doi.org/10.1007/BF01758773

[28] J. E. Hopcroft and R. E. Tarjan, "Algorithm 447: Efficient algorithms for graph manipulation," in *Communications of the ACM*, vol. 16, no. 6. ACM, 1973, pp. 372–378. [Online]. Available: https://doi.org/10.1145/362248.362272

[29] L. Wang, F. Yang, L. Zhuang, H. Cui, F. Lv, and X. Feng, "Articulation points guided redundancy elimination for betweenness centrality," *ACM SIGPLAN Notices*, vol. 51, no. 8, pp. 1–13, 2016. [Online]. Available: https://doi.org/10.1145/2851141.2851154

[30] J. Leskovec and A. Krevl, "SNAP Datasets: Stanford large network dataset collection," http://snap.stanford.edu/data, Jun. 2014.

[31] P. Kumar and H. H. Huang, "Graphone: A data store for real-time analytics on evolving graphs," *ACM Transactions on Storage (TOS)*, vol. 15, no. 4, pp. 1–40, 2020. [Online]. Available: https://doi.org/10.1145/3364180

[32] P. Fuchs, D. Margan, and J. Giceva, "Sortledton: A universal graph data structure," *ACM SIGMOD Record*, vol. 52, no. 1, pp. 17–25, 2023. [Online]. Available: https://doi.org/10.14778/3514061.3514065

[33] D. De Leo and P. Boncz, "Teseo and the analysis of structural dynamic graphs," *Proceedings of the VLDB Endowment*, vol. 14, no. 6, pp. 1053–1066, 2021. [Online]. Available: https://doi.org/10.14778/3447689.3447708

[34] X. Zhu, G. Feng, M. Serafini, X. Ma, J. Yu, L. Xie, A. Aboulnaga, and W. Chen, "Livegraph: A transactional graph storage system with purely sequential adjacency list scans," *arXiv preprint arXiv:1910.05773*, 2019. [Online]. Available: https://doi.org/10.14778/3384345.3384351

[35] P. Macko, V. J. Marathe, D. W. Margo, and M. I. Seltzer, "Llama: Efficient graph analytics using large multiversioned arrays," in *2015 IEEE 31st International Conference on Data Engineering*. IEEE, 2015, pp. 363–374. [Online]. Available: https://doi.org/10.1109/ICDE.2015.7113298

[36] G. Ramalingam and T. Reps, "On the computational complexity of dynamic graph problems," *Theoretical Computer Science*, vol. 158, no. 1-2, pp. 233–277, 1996. [Online]. Available: https://doi.org/10.1016/0304-3975(95)00079-8

[37] K. Vora, R. Gupta, and G. Xu, "Kickstarter: Fast and accurate computations on streaming graphs via trimmed approximations," in *Proceedings of the twenty-second international conference on architectural support for programming languages and operating systems*, 2017, pp. 237–251. [Online]. Available: https://doi.org/10.1145/3037697.3037748

[38] W. Fan, C. Tian, R. Xu, Q. Yin, W. Yu, and J. Zhou, "Incrementalizing graph algorithms," in *Proceedings of the 2021 International Conference on Management of Data*, 2021, pp. 459–471. [Online]. Available: https://doi.org/10.1145/3448016.3452796

[39] R. Bauer and D. Wagner, "Batch dynamic single-source shortest-path algorithms: An experimental study," in *International Symposium on Experimental Algorithms*. Springer, 2009, pp. 51–62. [Online]. Available: https://doi.org/10.1007/978-3-642-02011-7_7

[40] D. Nguyen, A. Lenharth, and K. Pingali, "A lightweight infrastructure for graph analytics," in *Proceedings of the twenty-fourth ACM symposium on operating systems principles*, 2013, pp. 456–471. [Online]. Available: https://doi.org/10.1145/2517349.2522739

[41] S. Hong, H. Chafi, E. Sedlar, and K. Olukotun, "Green-marl: a dsl for easy and efficient graph analysis," in *Proceedings of the seventeenth international conference on Architectural Support for Programming Languages and Operating Systems*, 2012, pp. 349–362. [Online]. Available: https://doi.org/10.1145/2150976.2151013

[42] X. Zhu, W. Chen, W. Zheng, and X. Ma, "Gemini: A {Computation-Centric} distributed graph processing system," in *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*, 2016, pp. 301–316. [Online]. Available: https://dl.acm.org/doi/10.5555/3026877.3026901

[43] G. Malewicz, M. H. Austern, A. J. Bik, J. C. Dehnert, I. Horn, N. Leiser, and G. Czajkowski, "Pregel: a system for large-scale graph processing," in *Proceedings of the 2010 ACM SIGMOD International Conference on Management of data*, 2010, pp. 135–146. [Online]. Available: https://doi.org/10.1145/1807167.1807184

[44] M. A. Bender and H. Hu, "An adaptive packed-memory array," *ACM Transactions on Database Systems (TODS)*, vol. 32, no. 4, pp. 26–es, 2007. [Online]. Available: https://doi.org/10.1145/1292609.1292616

[45] J. King, T. Gilray, R. M. Kirby, and M. Might, "Dynamic sparse-matrix allocation on gpus," in *International Conference on High Performance Computing*. Springer, 2016, pp. 61–80. [Online]. Available: https://doi.org/10.1007/978-3-319-41321-1_4

[46] A. A. R. Islam and D. Dai, "Dgap: Efficient dynamic graph analysis on persistent memory," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2023, pp. 1–13. [Online]. Available: https://doi.org/10.1145/3581784.3607106

[47] S. Brin and L. Page, "The anatomy of a large-scale hypertextual web search engine," *Computer Networks and ISDN Systems*, vol. 30, no. 1-7, pp. 107–117, 1998. [Online]. Available: https://doi.org/10.1016/S0169-7552(98)00110-X

[48] X. Zhu, Z. Ghahramani, and J. Lafferty, "Semi-supervised learning using gaussian fields and harmonic functions," in *Proceedings of the 20th International Conference on Machine Learning*, 2003, pp. 912–919. [Online]. Available: https://doi.org/10.5555/3041838.3041953